

Basics of Python Programming

Topics

- Python syntax and semantics
- Variables and data types
- Operators and logical evaluation
- Conditional statements and loops
- Lists, tuples, dictionaries, sets and strings
- Functions
- Classes and objects
- Regular expressions

Learning Objectives

By the end of this lecture, students should be able to:

- Explain the basic structure and syntax of a Python program.
- Create and use variables with appropriate data types.
- Apply arithmetic, comparison and logical operators.
- Use `if`, `elif`, `else` and loops.
- Select suitable Python data structures for different problems.
- Define and call functions.
- Explain the basic concepts of classes and objects.
- Perform simple pattern matching using regular expressions.

1. Introduction to Python

What is Python?

Python is a:

- **High-level** programming language
- **General-purpose** programming language
- **Interpreted** programming language
- **Dynamically typed** language
- **Object-oriented** language
- Language with a large standard library

Why Python?

Simple syntax



Easy to learn



Fast development



Large ecosystem



Web • AI • Data Science • Automation • Security

Why is Python Popular?

Major advantages

- Readable and beginner-friendly syntax
- Fewer lines of code for many tasks
- Cross-platform
- Huge ecosystem of libraries
- Supports multiple programming paradigms
- Excellent for scripting and automation
- Widely used in AI and data science

Example `print("Hello, Python!")` One statement is enough to display output.

Python Program: First Look

```
name = "Aman"  
age = 20  
  
print("Name:", name)  
print("Age:", age)
```

What happens?

1. "Aman" is assigned to name .
2. 20 is assigned to age .
3. print() displays the values.

Output

```
Name: Aman  
Age: 20
```

2. Python Syntax and Semantics

Syntax vs Semantics

Syntax

Rules that determine how code must be written.

Semantics

The meaning or behavior of the code.

Example:

```
x = 10
```

- Syntax: valid assignment statement.
- Semantics: the name `x` refers to the integer object `10`.

Python Indentation

Python uses **indentation** to define blocks of code.

```
age = 20

if age >= 18:
    print("Adult")
    print("Can vote")
```

The indented statements belong to the `if` block.

Incorrect

```
if age >= 18:  
print("Adult")
```

This causes an `IndentationError`.

Indentation: Think in Blocks

```
if condition:  
    |  
    | statement 1  
    | statement 2  
    | statement 3  
  
next statement
```

Example:

```
temperature = 35

if temperature > 30:
    print("Hot")
    print("Drink water")

print("Program continues")
```

The last `print()` is outside the `if` block.

Comments in Python

Comments explain code and are ignored by Python.

Single-line comment

```
# This is a comment  
age = 20
```

Inline comment

```
age = 20 # Student age
```

Why use comments?

- Explain complex logic
- Improve readability
- Help maintain programs
- Document assumptions

Python Identifiers

An **identifier** is a name used for variables, functions, classes, etc.

Valid

```
name  
student_name  
marks2  
_total
```

Invalid

```
2marks          # Cannot start with a digit  
student-name    # '-' is an operator  
class          # Reserved keyword
```


Naming convention

Use `snake_case` :

```
student_name  
total_marks  
average_score
```

Python Keywords

Keywords have special meanings in Python.

Examples:

```
if      else      elif
for     while     break
def     return    class
try     except    import
True    False     None
and     or        not
in      is
```

You cannot use a keyword as an ordinary variable name.

```
class = 10    # Invalid
```

3. Variables

What is a Variable?

A variable is a name that refers to a value/object.

```
age = 20  
name = "Rahul"
```

Think of:

```
age  -----> 20  
name -----> "Rahul"
```

Python does not require a separate declaration such as:

```
int age;
```

Instead:

```
age = 20
```

Dynamic Typing

Python determines the type at runtime.

```
x = 10
print(type(x))

x = "Python"
print(type(x))
```

Output:

```
<class 'int'>
<class 'str'>
```

The same variable name can refer to objects of different types.

Important

Dynamic typing does **not** mean Python has no types.

Python is strongly typed, but types are determined dynamically.

Multiple Assignment

Python allows convenient assignment.

```
x = 10  
y = 20  
z = 30
```

Can become:

```
x, y, z = 10, 20, 30
```

Swapping values

```
a = 10
```

```
b = 20
```

```
a, b = b, a
```

Now:

```
a = 20
```

```
b = 10
```


Constants

Python does not enforce true constants at the language level.

Convention:

```
PI = 3.14159  
MAX_USERS = 100
```

Uppercase names communicate:

|"Do not change this value."

It is a **programming convention**, not a restriction.

4. Python Data Types

Common built-in types:

Type	Example
<code>int</code>	<code>25</code>
<code>float</code>	<code>3.14</code>
<code>complex</code>	<code>2 + 3j</code>
<code>bool</code>	<code>True</code>
<code>str</code>	<code>"Python"</code>

Type	Example
list	[10, 20, 30]
tuple	(10, 20, 30)
dict	{"name": "Aman"}
set	{10, 20, 30}
NoneType	None

Numeric Data Types

Integer

```
age = 20
```

Floating point

```
price = 99.50
```

Complex

```
z = 2 + 3j
```

Check a type:

```
print(type(age))  
print(type(price))  
print(type(z))
```

Boolean Data Type

Boolean values:

```
True  
False
```

Example:

```
is_logged_in = True  
has_permission = False
```

Boolean expressions are common in decision-making:

```
age >= 18
```

Result:

True

Type Conversion

Python provides functions for conversion.

```
x = "25"  
  
age = int(x)  
price = float("99.50")  
text = str(100)
```


Common conversions:

```
int()  
float()  
str()  
bool()  
list()  
tuple()  
set()
```

Important

Not every conversion is valid:

```
int("hello")
```

causes a `ValueError`.

Input and Output

Output

```
print("Hello")
```

Input

```
name = input("Enter your name: ")  
print("Hello", name)
```

Important

`input()` returns a **string**.

```
age = input("Enter age: ")
```

If arithmetic is required:

```
age = int(input("Enter age: "))
```

Formatted Output: f-Strings

f-strings make formatted output easy.

```
name = "Aman"  
age = 20  
  
print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Aman and I am 20 years old.
```

Expressions can also be used:

```
a = 10  
b = 20  
  
print(f"Sum = {a + b}")
```

5. Operators

Operators perform operations on values.

Major categories

- Arithmetic
- Comparison
- Assignment
- Logical
- Membership
- Identity
- Bitwise

Arithmetic Operators

Operator	Meaning	Example
+	Addition	10 + 3
-	Subtraction	10 - 3
*	Multiplication	10 * 3
/	Division	10 / 3
//	Floor division	10 // 3
%	Modulus	10 % 3
**	Power	10 ** 3

Example

```
a = 10
b = 3

print(a + b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```


Division vs Floor Division

```
print(10 / 3)  
print(10 // 3)
```

Output:

```
3.3333333333333335  
3
```

/

Returns normal division result.

//

Returns floor division result.

For negative numbers, remember that floor means rounding toward negative infinity.

```
print(-10 // 3)
```

Comparison Operators

Comparison produces `True` or `False`.

Operator	Meaning
<code>==</code>	Equal
<code>!=</code>	Not equal
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal
<code><=</code>	Less than or equal

Example:

```
age = 20  
  
print(age >= 18)  
print(age == 20)  
print(age < 18)
```

= **VS** **==**

This is a common beginner mistake.

Assignment

```
x = 10
```

Means:

| Assign 10 to **x**.

Comparison

```
x == 10
```

Means:

| Is `x` equal to 10?

They have completely different purposes.

Logical Operators

Python provides:

```
and  
or  
not
```

and

True only when both conditions are true.

```
age >= 18 and citizen == True
```

or

True when at least one condition is true.

```
is_admin or is_teacher
```

not

Reverses a Boolean value.

```
not is_logged_in
```


Truth Table: **and**

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Example:

```
age = 25  
has_id = True  
  
print(age >= 18 and has_id)
```

Result:

```
True
```

Truth Table: **or**

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Example:

```
is_admin = False  
is_teacher = True  
  
print(is_admin or is_teacher)
```

Result:

```
True
```

Operator Precedence

Consider:

```
result = 10 + 5 * 2
```

Multiplication happens first:

```
10 + (5 × 2)  
= 20
```

Use parentheses when clarity matters:

```
result = (10 + 5) * 2
```

General idea

Parentheses



Exponentiation



Multiplication / Division



Addition / Subtraction



Comparisons



not



and



or

Membership Operators

Use:

```
in  
not in
```

Example:

```
name = "Python"  
  
print("P" in name)  
print("z" in name)  
print("z" not in name)
```

For lists:

```
numbers = [10, 20, 30]  
print(20 in numbers)
```


Identity Operators

Use:

```
is  
is not
```

Identity checks whether two references refer to the **same object**.

```
a = [1, 2]  
b = a  
c = [1, 2]  
  
print(a == c) # Same contents  
print(a is c) # Different objects  
print(a is b) # Same object
```

Remember

```
==  → equality  
is  → identity
```

6. Conditional Statements

Programs often need to make decisions.



Python provides:

```
if  
elif  
else
```

if Statement

Syntax:

```
if condition:  
    statement
```

Example:

```
age = 20  
  
if age >= 18:  
    print("Adult")
```

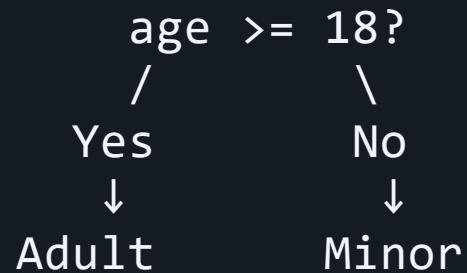
If the condition is **True**, the block executes.

if-else

```
age = 16
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Exactly one branch executes.

Flow



if-elif-else

Use multiple conditions.

```
marks = 72

if marks >= 90:
    grade = "A+"
elif marks >= 75:
    grade = "A"
elif marks >= 60:
    grade = "B"
else:
    grade = "C"

print(grade)
```

Conditions are evaluated from top to bottom.

Nested Conditions

An `if` can contain another `if`.

```
age = 25
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Underage")
```

Tip

Avoid excessive nesting when a simpler logical expression is clearer.

7. Loops

Loops repeat instructions. wo primary loops:

```
for  
while
```

Use loops when:

- Repeating an operation
- Processing collections
- Counting
- Searching
- Reading items one by one

for Loop

A `for` loop iterates over an iterable.

```
for number in [1, 2, 3, 4]:  
    print(number)
```

Output:

```
1  
2  
3  
4
```

Python's `for` loop is conceptually:

"For each item in this collection..."

range()

`range()` is frequently used with `for`.

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Forms

```
range(stop)  
range(start, stop)  
range(start, stop, step)
```

The `stop` value is excluded.

Examples of `range()`

```
range(5)
```

```
# 0 1 2 3 4
```

```
range(2, 6)
```

```
# 2 3 4 5
```

```
range(2, 10, 2)
```

```
# 2 4 6 8
```

```
range(10, 0, -2)
```

```
# 10 8 6 4 2
```

while Loop

A `while` loop repeats while a condition is true.

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

Output:

```
1
2
3
```

Important

Make sure the loop condition eventually becomes false.

Infinite Loop

This loop never ends:

```
count = 1
while count <= 5:
    print(count)
```

Why?

`count` never changes.

Correct

```
while count <= 5:
    print(count)
    count += 1
```

break

`break` immediately terminates a loop.

```
for i in range(1, 11):  
    if i == 4:  
        break  
    print(i)
```

Output:

```
1  
2  
3
```

Use `break` when the desired result has been found or the loop must stop.

continue

`continue` skips the current iteration.

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Output:

```
1  
2  
4  
5
```


pass

`pass` does nothing.

It is useful as a placeholder.

```
for i in range(5):  
    pass
```

Also:

```
def future_function():  
    pass
```

Difference

```
break    → stop loop  
continue → skip current iteration  
pass     → do nothing
```

Loop with `else`

Python loops can have an `else`.

```
for i in range(5):  
    print(i)  
else:  
    print("Loop completed")
```

The `else` runs when the loop finishes normally.

If `break` occurs, the loop's `else` is skipped.

Example: Prime Check

```
n = 17

if n < 2:
    print("Not prime")
else:
    for i in range(2, n):
        if n % i == 0:
            print("Not prime")
            break
    else:
        print("Prime")
```

Idea

A prime number has no divisor other than:

`1 and itself`

8. Strings

A string is a sequence of characters.

```
name = "Python"  
message = 'Hello World'
```

Strings are:

- Ordered
- Indexable
- Sliceable
- Immutable

String Indexing

```
text = "Python"
```

Indexes:

P	y	t	h	o	n
0	1	2	3	4	5

Negative indexes:

P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

Example:

```
print(text[0])  
print(text[-1])
```

Output:

```
p  
n
```


String Slicing

Syntax:

```
text[start:stop:step]
```

Example:

```
text = "Python"  
  
print(text[0:3])  
print(text[2:5])  
print(text[::-1])
```

Output:

```
Pyt  
tho  
nohtyP
```

Remember:

The `stop` index is excluded.

Useful String Methods

```
text = " Python Programming "  
  
print(text.strip())  
print(text.lower())  
print(text.upper())  
print(text.replace("Python", "Java"))  
print(text.split())
```

Other useful methods:

```
find()  
startswith()  
endswith()  
count()  
join()
```

String Immutability

Strings cannot be modified in place.

```
text = "Python"  
  
# text[0] = "J"    # TypeError
```

Instead, create a new string:

```
text = "J" + text[1:]  
print(text)
```

Output:

```
Jython
```

9. Lists

A list stores an ordered collection of items. `numbers = [10, 20, 30, 40]`

Lists can contain different types:

```
data = [10, "Python", 3.14, True]
```

Properties

- Ordered
- Mutable
- Allows duplicates
- Supports indexing and slicing

List Operations

```
numbers = [10, 20, 30]  
  
numbers.append(40)  
numbers.insert(1, 15)  
numbers.remove(20)  
  
print(numbers)
```

Common methods:

```
append()  
insert()  
remove()  
pop()  
sort()  
reverse()  
clear()
```

List Indexing and Slicing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])
print(numbers[-1])
print(numbers[1:4])
```

Output:

```
10
50
[20, 30, 40]
```


Lists can be modified:

```
numbers[0] = 100
```

List Comprehension

A compact way to create lists.

Traditional:

```
squares = []  
  
for x in range(1, 6):  
    squares.append(x * x)
```

List comprehension:

```
squares = [x * x for x in range(1, 6)]
```

Result:

```
[1, 4, 9, 16, 25]
```

Conditional List Comprehension

```
even_numbers = [  
    x for x in range(1, 11)  
    if x % 2 == 0  
]
```

Result:

```
[2, 4, 6, 8, 10]
```

Pattern:

```
[expression for item in iterable if condition]
```

Use comprehensions when they improve readability.

10. Tuples

A tuple is an ordered collection that is **immutable**.

```
coordinates = (10, 20)
```

Properties

- Ordered
- Immutable
- Allows duplicates
- Supports indexing and slicing

Example:

```
student = ("Aman", 20, "CSE")
```

Tuple Unpacking

```
student = ("Aman", 20, "CSE")  
  
name, age, branch = student  
  
print(name)  
print(age)  
print(branch)
```

Output:

```
Aman  
20  
CSE
```

This is called **unpacking**.

List vs Tuple

Feature	List	Tuple
Syntax	[]	()
Mutable	Yes	No
Ordered	Yes	Yes
Duplicates	Yes	Yes
Indexing	Yes	Yes
Typical use	Changeable data	Fixed data

Example:

```
shopping_cart = ["Book", "Pen"]  
coordinates = (28.6, 77.2)
```

11. Dictionaries

A dictionary stores data as **key-value pairs**.

```
student = {  
    "name": "Aman",  
    "age": 20,  
    "branch": "CSE"  
}  
  
print(student["name"])
```

Output:

```
Aman
```

Dictionary Operations

```
student["age"] = 21  
student["city"] = "Delhi"  
  
print(student)
```

Useful methods:

```
keys()  
values()  
items()  
get()  
update()  
pop()
```

Example:

```
print(student.get("name"))
```

`get()` is useful when a key may not exist.

```
data = {"name": "Alice", "age": 25}
```

- Using `get()` when key exists

```
print(data.get("name"))      # Output: Alice
```

- Using `get()` when key does not exist

```
print(data.get("city"))      # Output: None
```

- Providing a default value if key is missing

```
print(data.get("city", "Unknown")) # Output: Unknown
```

Why get() is useful:

- Avoids KeyError when the key is missing.
 - Allows a default value to be returned instead of None.
 - Makes code cleaner than using if key in dict checks.
- ✓ Best practice: Use get() when you're not sure if a key exists and you want a safe, concise way to handle missing keys.

Dictionary Iteration

```
student = {  
    "name": "Aman",  
    "age": 20  
}  
  
for key, value in student.items():  
    print(key, ":", value)
```

Output:

```
name : Aman  
age : 20
```

Dictionary comprehensions are also possible:

```
squares = {x: x*x for x in range(1, 5)}
```

Output

squares

{1: 1, 2: 4, 3: 9, 4: 16}

12. Sets

A set stores a collection of **unique elements**.

```
numbers = {10, 20, 30, 20}  
  
print(numbers)
```

Duplicate `20` is removed.

Properties

- Unique elements
- Mutable set object
- No indexing
- Useful for membership testing and set operations

Set Operations

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)    # Union
print(A & B)    # Intersection
print(A - B)    # Difference
print(A ^ B)    # Symmetric difference
```

Conceptually:

```
A ∪ B → Union
A ∩ B → Intersection
A - B → Difference
A △ B → Symmetric difference
```

Feature	<code>difference()</code>	<code>symmetric_difference()</code>
Logic	Items in A but NOT in B	Items in A or B but NOT in both
Operator	<code>-</code>	<code>^</code>
Method	<code>A.difference(B)</code>	<code>A.symmetric_difference(B)</code>
Order Dependent	Yes (<code>A - B</code> $\neq$ <code>B - A</code>)	No (<code>A ^ B</code> $==$ <code>B ^ A</code>)
Result for A & B	<code>{1, 2}</code>	<code>{1, 2, 4, 5}</code>

Choosing a Data Structure

Requirement	Recommended
Text	String
Ordered, changeable collection	List
Ordered, fixed collection	Tuple
Key-value mapping	Dictionary
Unique values	Set

Example

```
names = ["Aman", "Ravi", "Aman"]      # duplicates allowed
point = (10, 20)                       # fixed
student = {"name": "Aman"}             # key-value
ids = {101, 102, 103}                  # unique
```

13. Functions

A function is a reusable block of code.

Instead of repeating:

```
print("Hello")  
print("Hello")
```

Define:

```
def greet():  
    print("Hello")
```

Call it:

```
greet()
```

Benefits

- Reusability
- Modularity
- Easier testing
- Better readability
- Easier maintenance

Function Syntax

```
def function_name(parameters):  
    statements
```

Example:

```
def add(a, b):  
    result = a + b  
    return result
```

Call:

```
answer = add(10, 20)  
print(answer)
```

Output:

```
30
```


Parameters and Arguments

```
def greet(name):  
    print("Hello", name)
```

Here:

name → parameter

Call:

```
greet("Aman")
```

Here:

"Aman" → argument

Return Values

A function can return a result.

```
def square(x):  
    return x * x  
  
result = square(5)  
print(result)
```

Output:

```
25
```

return

- Sends a value back to the caller.
- Ends execution of that function.

Default Arguments

```
def greet(name="Student"):  
    print("Hello", name)
```

Calls:

```
greet("Aman")  
greet()
```

Output:

```
Hello Aman  
Hello Student
```

Default values are used when the argument is omitted.

Keyword Arguments

```
def student_info(name, age):  
    print(name, age)  
  
student_info(age=20, name="Aman")
```

Keyword arguments make the intended parameter explicit.

Important

Positional arguments generally come before keyword arguments.

Variable-Length Arguments

***args**

Accepts multiple positional arguments.

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(10, 20, 30))
```

****kwargs**

Accepts multiple keyword arguments.

```
def show(**data):  
    print(data)  
  
show(name="Aman", age=20)
```

```
{'name': 'Aman', 'age': 20}
```

Scope of Variables

Local variable

Defined inside a function.

```
def test():  
    x = 10  
    print(x)
```

`x` is local to `test()`.

Global variable

Defined outside functions.

```
x = 100

def show():
    print(x)
```

Python follows scope rules to determine which name a variable refers to.

Lambda Functions

A lambda is a small anonymous function.

```
square = lambda x: x * x  
  
print(square(5))
```

Output: 25

Equivalent regular function:

```
def square(x):  
    return x * x
```

Use lambdas when a short function improves clarity.

14. Object-Oriented Programming

What is OOP?

Object-Oriented Programming organizes programs around **objects**.

An object can contain:

- Data → attributes
- Behavior → methods

Real-world analogy

```
Student
├── name
├── age
└── study()
```

A Python object can model the same idea.

Class vs Object

Class: A blueprint/template.

```
Class: Student
      /   |   \
    name age study()
```

Object

A specific instance of a class.

```
student1 → Aman, 20
student2 → Ravi, 21
```

One class can create many objects.

Creating a Class

```
class Student:  
    pass
```

Create an object:

```
s1 = Student()  
s2 = Student()
```

Here:

```
Student → class  
s1       → object  
s2       → object
```

Attributes

Attributes store object data.

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Create object:

```
s1 = Student("Aman", 20)  
  
print(s1.name)  
print(s1.age)
```

The `__init__()` Method

`__init__()` is commonly used to initialize an object.

```
class Student:  
    def __init__(self, name):  
        self.name = name
```

When:

```
s1 = Student("Aman")
```

Python initializes the object using the supplied value.

What is `self`?

`self` refers to the **current object instance**.

```
class Student:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(self.name)
```

`self` in Python and `this` in C++ are conceptually similar but not exactly the same.

- Both refer to the current instance of the class.
- Both are used to access instance variables and methods from within the class.

Call:

```
s1 = Student("Aman")  
s1.display()
```

`self.name` means:

The `name` attribute belonging to this object.

Methods

Methods are functions defined inside a class.

```
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
calc = Calculator()  
  
print(calc.add(10, 20))
```

Output:

```
30
```

Objects use methods to perform behavior.

Instance Attributes

Different objects can contain different values.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
s1 = Student("Aman", 85)
s2 = Student("Ravi", 92)
```

Now:

```
s1.name    → Aman
s1.marks   → 85
s2.name    → Ravi
s2.marks   → 92
```

Basic Inheritance

Inheritance allows a class to derive from another class.

```
class Animal: # [Parent class]
    def speak(self):
        print("Animal sound")

class Dog(Animal): # Inheritance implemented here [Child class]
    def bark(self):
        print("Woof!")
```

Create:

```
dog = Dog()  
  
dog.speak()  
dog.bark()
```

`Dog` inherits the `speak()` method from `Animal`.

Method Overriding [Polymorphism]

A child class can provide its own implementation.

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Woof!")
```

Now:

```
dog = Dog()  
dog.speak()
```

Output:

```
Woof!
```

This is an example of polymorphic behavior.

OOP: Core Concepts

- Encapsulation: Keep data and behavior together.
- Inheritance: Create a new class from an existing class.
- Polymorphism: Different objects can respond to the same operation differently.
- Abstraction: Expose essential behavior while hiding unnecessary implementation details.

For beginners:

```
Class → Blueprint  
Object → Instance  
Attribute → Data  
Method → Behavior
```

15. Regular Expressions

What is a Regular Expression?

A **regular expression** (regex) is a pattern used to search, match, validate or extract text.

Common uses:

- Email validation
- Phone-number patterns
- Searching text
- Extracting dates
- Finding keywords
- Replacing text

Python provides the `re` module.

```
import re
```

Basic Regex Symbols

Pattern	Meaning
.	Any character except newline
^	Start of string
\$	End of string
*	Zero or more
+	One or more
?	Zero or one
{m,n}	Between m and n

Pattern	Meaning
[]	Character set
()	Group
\d	Digit
\w	Word character
\s	Whitespace

re.search()

Searches for a match anywhere in the string.

```
import re

text = "Python is powerful"

result = re.search("Python", text)

if result:
    print("Found")
```

Output:

```
Found
```

`re.match()`

`match()` checks from the **beginning** of the string.

```
import re

text = "Python programming"

result = re.match("Python", text)

if result:
    print("Starts with Python")
```

Compare:

```
match() → beginning
search() → anywhere
```

re.findall()

Finds all non-overlapping matches.

```
import re

text = "My numbers are 10, 20 and 30."

numbers = re.findall(r"\d+", text) # ` \d ` : Digit, + : One or more occurrence

print(numbers)
```


Output:

```
['10', '20', '30']
```

The `r` before the string creates a raw string, which is especially useful for regex patterns.

It treats backslashes () as literal characters instead of escape characters

Regex Character Classes

- Digits: `\d`
- Non-digit: `\D`
- Word character: `\w`
- Whitespace: `\s`
- Custom character set:
 - `regex[aeiou]` Matches one vowel.
 - `regex[0-9]` Matches one digit.

Regex Quantifiers

Suppose we want one or more digits:

```
\d+
```

Examples:

```
7  
42  
12345
```

Zero or more:

```
\d*
```

Exactly 4 digits:

```
\d{4}
```

Between 2 and 5 digits:

```
\d{2,5}
```

Example: Simple Phone Pattern

```
import re

phone = "9876543210"
pattern = r"^\d{10}$"

if re.fullmatch(pattern, phone):
    print("Valid phone number")
else:
    print("Invalid phone number")
```

The pattern means:

<code>^</code>	start
<code>\d{10}</code>	exactly 10 digits
<code>\$</code>	end

Function	Match Location	Requirement	Equivalent Pattern
<code>re.match()</code>	Starts at the beginning	Only the <i>prefix</i> needs to match.	<code>r"pattern"</code> (effectively acts like <code>r"^pattern"</code>)
<code>re.fullmatch()</code>	Starts at the beginning	The <i>entire string</i> must match from start to finish.	<code>r"^pattern\$"</code>

Example: Simple Email Pattern

```
import re

email = "student@example.com"

pattern = r"^[\\w.-]+@[\\w.-]+\\. [A-Za-z]{2,}$"

if re.fullmatch(pattern, email):
    print("Valid format")
else:
    print("Invalid format")
```

Note

Regex validation can check a **format**. It cannot prove that an email address actually exists.

Explanation

```
r"^[ \w.- ]+@[ \w.- ]+\.[A-Za-z]{2,}$"
```

- `r"..."`: Marks the string as a raw string in Python. This prevents Python from interpreting backslashes (like `\w`) as escape characters, passing them directly to the `re` module.
- `^`: The start anchor. It forces the match to begin at the very start of the string, preventing matches that have extra text at the beginning.


```
r"^[\w.-]+@[ \w.-]+\.[A-Za-z]{2,}$"
```

- `[\w.-]+`: Matches the username part of the email.
 - `[...]` is a character set (matches any character inside it).
 - `\w` matches any alphanumeric character (A-Z, a-z, 0-9) and underscores (`_`).
 - `.` and `-` match literal periods and hyphens.
- `+` means one or more of these characters are required.
- `@`: Matches the literal `@` symbol separating the username and domain.

```
r"^[\w.-]+@[ \w.-]+\.[A-Za-z]{2,}$"
```

- `[\w.-]+`: Matches the domain name (e.g., gmail, yahoo, or sub.domain).
 - It uses the same rules as the username part and requires one or more characters.
 - `\.`: Matches a literal period (.).
 - The backslash is necessary because a standalone dot `.` in regex means "any character".

```
r"^[\w.-]+@[\w.-]+\.[A-Za-z]{2,}$"
```

- `[A-Za-z]{2,}`: Matches the Top-Level Domain (TLD) (e.g., com, org, edu, co.uk).
- `[A-Za-z]` restricts this part to letters only (uppercase or lowercase).
- `{2,}` is a quantifier meaning at least 2 characters long, with no upper limit.
- `$`: The end anchor.
 - It forces the match to end at the very end of the string, preventing trailing text.

Regex: `sub()`

`re.sub()` replaces matches.

```
import re

text = "Python 123 is 456"

result = re.sub(r"\d+", "#", text)

print(result)
```

Output:

```
Python # is #
```

Useful for:

- Cleaning text
- Masking sensitive patterns
- Standardizing data

Mini Project: Student Result

Combine variables, input, conditions and functions.

```
def calculate_grade(marks):  
    if marks >= 90:  
        return "A+"  
    elif marks >= 75:  
        return "A"  
    elif marks >= 60:  
        return "B"  
    elif marks >= 40:  
        return "C"  
    else:  
        return "F"  
  
marks = int(input("Enter marks: "))  
print("Grade:", calculate_grade(marks))
```

Concepts used:

- Function
- Input
- Integer conversion
- Conditions
- Return value

Mini Project: Count Vowels

```
text = input("Enter text: ")

count = 0

for ch in text.lower():
    if ch in "aeiou":
        count += 1

print("Vowels:", count)
```


Concepts

- String
- `input()`
- `for`
- `if`
- Membership operator `in`
- Counter variable

Mini Project: Word Frequency

```
text = "python is easy and python is powerful"

words = text.split()
frequency = {}

for word in words:
    frequency[word] = frequency.get(word, 0) + 1

print(frequency)
```

If word does not exist, it safely returns the fallback value 0 instead of throwing a `KeyError`

Possible output:

```
{  
  'python': 2,  
  'is': 2,  
  'easy': 1,  
  'and': 1,  
  'powerful': 1  
}
```

Concepts:

- String
- List
- Dictionary
- Loop
- `get()`

Common Beginner Mistakes

1. Forgetting indentation

```
if x > 10:  
print(x)
```

2. Using `=` instead of `==`

```
if x = 10:    # Wrong
```

3. Forgetting type conversion

```
age = input("Age: ")  
# age is a string
```

4. Infinite **while** loop

Always update the loop state when necessary.

```
# 1. Initialize a variable
count = 1

# 2. Set the condition
while count <= 5:
    print(f"Iteration number: {count}")
    # 3. Update the variable (crucial step!)
    count += 1
```

An f-string (formatted string literal) is the modern, standard, and most efficient way to format strings in Python

Common Beginner Mistakes — Continued

5. Index out of range

```
numbers = [10, 20]  
  
print(numbers[5])
```

6. Modifying an immutable string

```
text = "Python"  
  
# text[0] = "J" # Error
```

7. Confusing `==` and `is`

Use:

```
==    # compare values  
is    # compare object identity
```

8. Forgetting `return`

A function that calculates a result must return it if the caller needs that result.

Practice Exercise 1

Write a program to:

1. Read two numbers.
2. Calculate:
 - Sum
 - Difference
 - Product
 - Quotient
 - Remainder
3. Display the results.

Challenge

Handle division by zero safely.

Solution

```
n=5
d=0
res=0
try:
    res=n/d
    print(res)
except ZeroDivisionError:
    print("Warning: Attempted to divide by zero.")
```

Practice Exercise 2

Write a program to read marks and display:

```
90-100 → A+  
75-89  → A  
60-74  → B  
40-59  → C  
Below 40 → F
```

Additional requirement

Reject marks below 0 or above 100 .

Practice Exercise 3

Write a program to print:

```
1  
2  
3  
...  
N
```

Then modify it to print:

```
N  
N-1  
...  
1
```

Use a `for` loop.

Then solve the same problem using a `while` loop.

Practice Exercise 4

Given:

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

Write programs to:

1. Find the largest value.
2. Find the smallest value.
3. Calculate the sum.
4. Count even numbers.
5. Create a list containing only values greater than 20.

Practice Exercise 5

Create a dictionary:

```
student = {  
    "name": "...",  
    "age": ...,  
    "marks": ...  
}
```


Write a program to:

1. Display all keys.
2. Display all values.
3. Calculate whether the student passed.
4. Add a "grade" key.
5. Print the complete dictionary.

Practice Exercise 6 — OOP

Create a class:

```
BankAccount
```

It should contain:

- `account_holder`
- `balance`

Methods:

```
deposit()  
withdraw()  
display_balance()
```

Rules:

- Deposit must be positive.
- Withdrawal must not exceed the balance.

Practice Exercise 7 — Regular Expressions

Using the `re` module:

1. Extract all numbers from a sentence.
2. Find all words beginning with `"P"`.
3. Check whether a string contains exactly 10 digits.
4. Replace all digits with `#`.
5. Count occurrences of a particular word.

- Quick Revision: Form the foundation for advanced Python programming.

Syntax + Semantics



Variables + Data Types



Operators



Conditions



Loops



Data Structures



Functions

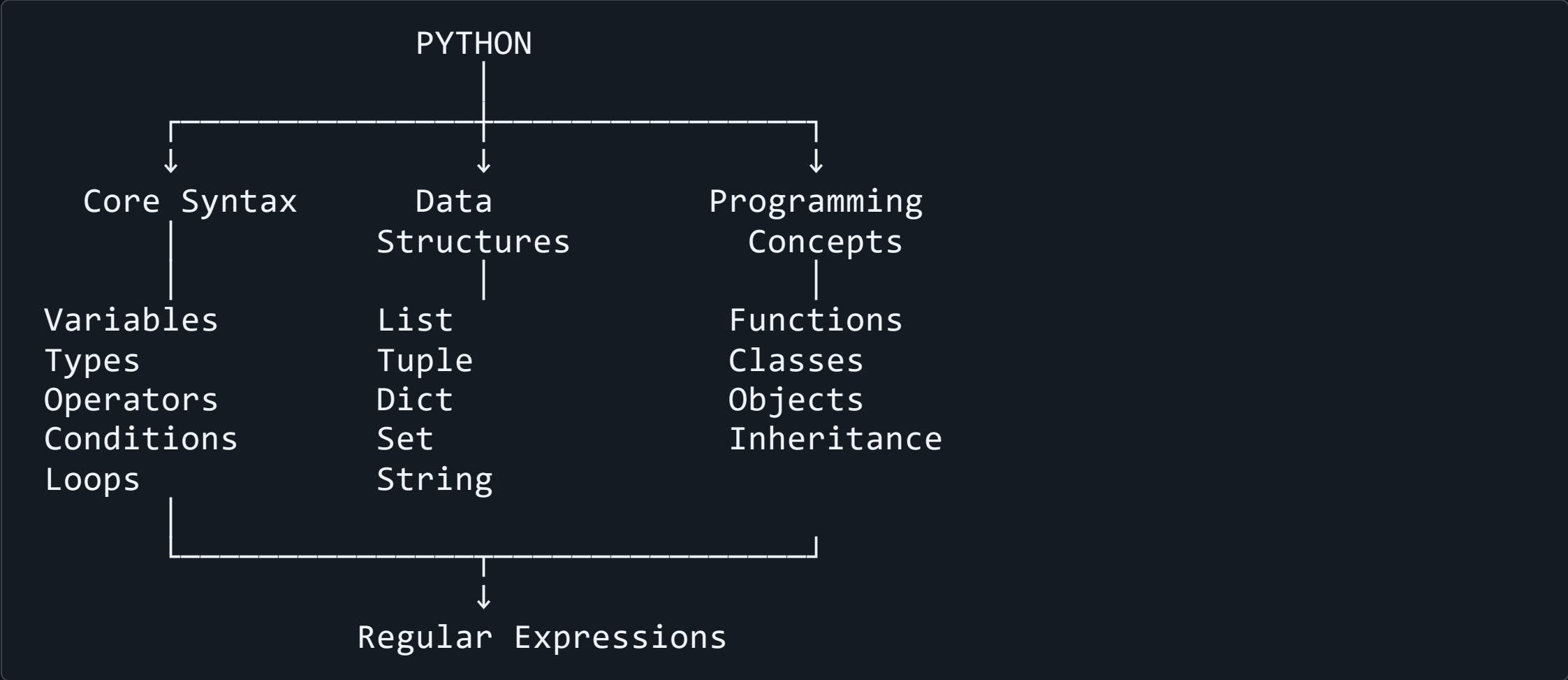


Classes + Objects



Regular Expressions

Concept Map



Key Takeaways

Remember these principles

- Indentation defines Python code blocks.
- Variables refer to objects and Python is dynamically typed.
- Comparison and logical operators produce Boolean results.
- `if`, `elif`, and `else` control decisions.
- `for` and `while` perform repetition.
- Lists are mutable; tuples are immutable.
- Dictionaries store key-value pairs.
- Sets store unique elements.

- Functions provide reusable behavior.
- Classes define object structure and behavior.
- Regular expressions provide pattern-based text processing.

Final Quick Quiz

- Q1: What is the difference between `=` and `==`?
- Q2: What is the output?

```
print(10 // 3)
```

- Q3: Which data structure stores key-value pairs?
- Q4: What does `break` do?
- Q5: What is the difference between a class and an object?
- Q6: Which module is used for regular expressions?
- Q7: What does `\d+` generally match?

End of Lecture

From Beginner to Python Programmer

Next Step

Practice by writing small programs.

Learn → Write → Run → Debug → Improve

Programming skill develops through practice, not memorization.

Appendix: Useful Python Cheat Sheet

```
# Output
print("Hello")
# Input
name = input("Name: ")
# Condition
if x > 10:
    print("Large")
# For loop
for i in range(5):
    print(i)
# While loop
while x > 0:
    x -= 1
# Function
def add(a, b):
    return a + b
```

```
# List
items = [1, 2, 3]

# Tuple
point = (10, 20)

# Dictionary
student = {"name": "Aman"}

# Set
unique = {1, 2, 3}

# Regex
import re
```

Appendix: Suggested Practice Path

Level 1 — Fundamentals

- Print statements
- Variables
- Data types
- Input/output
- Operators

Level 2 — Control Flow

- `if`
- `elif`
- `else`
- `for`
- `while`
- `break`
- `continue`

Level 3 — Data

- Strings
- Lists
- Tuples
- Dictionaries
- Sets

Level 4 — Programming Design

- Functions
- Scope
- Classes
- Objects
- Inheritance

Level 5 — Text Processing

- Regular expressions
- Searching
- Extraction
- Validation
- Replacement